

NoteLLM: A Retrievable Large Language Model for Note Recommendation

Chao Zhang

University of Science and
Technology of China &
State Key Laboratory of
Cognitive Intelligence
Hefei, China
zclfe00@gmail.com

Shiwei Wu

University of Science and
Technology of China &
State Key Laboratory of
Cognitive Intelligence
Hefei, China
dwustc@mail.ustc.edu.cn

Haoxin Zhang

Xiaohongshu Inc.
Beijing, China
zhanghaoxin@xiaohongshu.com

Tong Xu*

University of Science and
Technology of China &
State Key Laboratory of
Cognitive Intelligence
Hefei, China
tongxu@ustc.edu.cn

Yan Gao

Xiaohongshu Inc.
Beijing, China
yadun@xiaohongshu.com

Yao Hu

Xiaohongshu Inc.
Beijing, China
xiahou@xiaohongshu.com

Di Wu

Xiaohongshu Inc.
Beijing, China
wudi1123@foxmail.com

Enhong Chen*

University of Science and
Technology of China &
State Key Laboratory of
Cognitive Intelligence
Hefei, China
cheneh@ustc.edu.cn

ABSTRACT

People enjoy sharing "notes" including their experiences within online communities. Therefore, recommending notes aligned with user interests has become a crucial task. Existing online methods only input notes into BERT-based models to generate note embeddings for assessing similarity. However, they may underutilize some important cues, e.g., hashtags or categories, which represent the key concepts of notes. Indeed, learning to generate hashtags/categories can potentially enhance note embeddings, both of which compress key note information into limited content. Besides, Large Language Models (LLMs) have significantly outperformed BERT in understanding natural languages. It is promising to introduce LLMs into note recommendation. In this paper, we propose a novel unified framework called NoteLLM, which leverages LLMs to address the item-to-item (I2I) note recommendation. Specifically, we utilize Note Compression Prompt to compress a note into a single special token, and further learn the potentially related notes' embeddings via a contrastive learning approach. Moreover, we use NoteLLM to summarize the note and generate the hashtag/category automatically through instruction tuning. Extensive validations on real scenarios demonstrate the effectiveness of our proposed method compared with the online baseline and show major improvements in the recommendation system of Xiaohongshu.

*Corresponding authors.

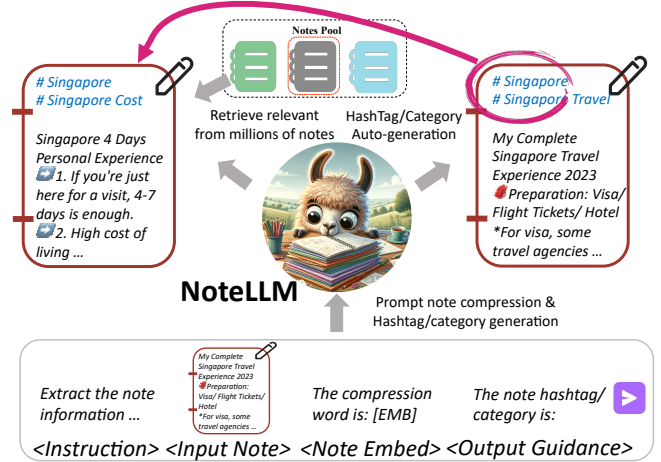


Figure 1: An example of recommending the relevant note from millions-level notes pool via NoteLLM. Learning hashtag generation benefits item-to-item recommendation tasks.

CCS CONCEPTS

• Information systems → Recommender systems.

KEYWORDS

Large Language Model; Recommendation; Hashtag Generation

ACM Reference Format:

Chao Zhang, Shiwei Wu, Haoxin Zhang, Tong Xu, Yan Gao, Yao Hu, Di Wu, and Enhong Chen. 2024. NoteLLM: A Retrievable Large Language Model for Note Recommendation. In *Companion Proceedings of the ACM Web Conference 2024 (WWW '24 Companion)*, May 13–17, 2024, Singapore, Singapore. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3589335.3648314>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW '24 Companion, May 13–17, 2024, Singapore, Singapore

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0172-6/24/05...\$15.00
<https://doi.org/10.1145/3589335.3648314>

1 INTRODUCTION

Focused on user-generated content (UGC) and providing a more authentic and personalized user experience, social media like Xiaohongshu and Lemon8 have gained significant popularity among users. These platforms encourage users to share their product reviews, travel blogs, and life experiences, among other content, also referred to as "notes". By providing more personalized notes based on user preferences, note recommendation plays a crucial part in enhancing user engagement [16, 34, 48, 64]. Item-to-item (I2I) note recommendation is a classic way to retrieve notes of potential interest to the user from the millions-level notes pool [19, 65]. Given a target note, I2I methods select the relevant notes according to content [65] or collaborative signals [19].

Existing online methods of I2I note recommendation usually input whole note content into BERT-based models [3] to generate embeddings of notes, and recommend relevant notes based on embedding similarity [11, 36]. However, these methods merely treat hashtags/categories as a component of note content, underutilizing their potential. As shown in Figure 1, hashtags/categories (e.g., #Singapore) represent the central ideas of notes, which are crucial in determining whether two notes contain related content. In fact, we find that generating hashtags/categories is similar to producing note embeddings. Both compress the key note information into limited content. Therefore, learning to generate hashtags/categories can potentially enhance the quality of embeddings. Besides, Large Language Models (LLMs) have recently exhibited powerful abilities in natural languages [10, 24, 42, 54] and recommendations [1, 2, 34, 59]. However, there is a scarcity of research investigating the application of LLMs in I2I recommendations. Utilizing LLMs to improve I2I note recommendations holds considerable promise.

Inspired by the above insights, we propose a unified multi-task approach called NoteLLM in this paper. Based on LLMs, NoteLLM learns from the I2I note recommendation and hashtag/category generation tasks, aiming to enhance the I2I note recommendation ability by learning to extract condensed concepts. Specifically, we first construct a unified Note Compression Prompt for each note sample and then decode via pre-trained LLMs (e.g., LLaMA 2 [42]), which utilize a special token to compress the note content and generate hashtags/categories simultaneously. To construct the related note pairs, we count the co-occurrence scores for all note pairs from user behaviours, and form the set of co-occurrence scores for each note. We select notes with the highest co-occurrence scores in the set as the related notes for a given note. Further, to recommend the relevant notes for each sample, Generative-Contrastive Learning (GCL) utilizes the compressed tokens as the embedding of each note, and then trains the LLMs to identify the related notes from in-batch negatives. Simultaneously, we employ Collaborative Supervised Fine-tuning (CSFT) approach to train models to generate hashtags/categories for each note. Since both the compression token learned by the I2I note recommendation task and the hashtag/category generation task aim to extract the key concept of the note content, CSFT can enhance note embeddings effectively.

Our paper makes the following contributions:

- To the best of our knowledge, our NoteLLM framework is the first to address the I2I recommendation task utilizing LLMs. It reveals

that introducing LLMs is a practical and promising strategy to enhance I2I recommendation systems.

- We propose a multi-task framework to learn I2I recommendation task and hashtag/category generation task to enhance note embeddings. We demonstrate that learning to generate the compressed concepts is beneficial to the I2I recommendation task.
- Extensive validations on offline experiments and online industrial scenarios of Xiaohongshu demonstrate the effectiveness of our proposed technical framework for note recommendation.

2 RELATED WORK

2.1 I2I Recommendation

I2I recommendation is a crucial technique that can recommend a ranked list of items from a large-scale item pool based on a target item. I2I recommendation either pre-constructs the I2I index [55] or retrieves relevant items online using the approximate k-nearest neighbor method [12]. Traditional I2I recommendations typically rely solely on collaborative signals from user behaviors [55, 67]. However, these methods cannot manage cold-start items due to lack of user-item interaction [65]. To address this issue, numerous studies have investigated content-based I2I recommendations [8, 65]. We focus on the text-based I2I recommendation system, which measures the similarity of items based on their textual content. Initially, representation of text-based I2I recommendation relied on a term-based sparse vector matching mechanism [35, 37]. With the advent of deep learning, neural networks have proven more adept at representing text information [3, 27]. Previous works [13, 25, 52, 53] transform texts into embeddings in the same latent space to measure their relationship through embedding similarity. LLMs have recently gained great attention for their impressive abilities [33, 54, 56]. However, the application of LLMs in I2I recommendation remains unexplored. Besides, some studies treat LLMs solely as encoders for generating embeddings [10, 26, 28], failing to leverage their full potential for generation. In NoteLLM, we utilize LLMs to generate hashtags/categories, which can enhance note embeddings.

2.2 LLMs for Recommendation

LLMs have recently made significant advancements [31, 41, 42]. Consequently, numerous studies incorporate LLMs into recommendation tasks [5, 18, 50]. There are three main methods of integrating LLMs with recommendations [18, 50]. The first method is utilizing LLMs to augment data [21, 29, 51]. Due to the abundant world knowledge contained by LLMs, the augmented data are more prominent and diverse than the raw data [23, 46, 51]. However, these methods require continuous preprocessing of the testing data to align with the augmented training data and are highly dependent on the quality of LLMs' generation. The second method is leveraging LLMs to recommend directly. These methods design special prompts [9, 20, 43] or use supervised finetuning [1, 2, 59] to induce LLMs to answer the given questions. Nevertheless, because of the limited context length, these methods only focus on the reranking stage [7, 59], which only contains dozens of candidate items. The last method is adopting LLMs as the encoders to generate embeddings representing specific items [15, 49]. Although these methods are effective to extract information, they all discard the generative

capabilities of LLMs. In contrast to above methods, NoteLLM employs LLMs during the recall phase and learns hashtag generation to improve LLMs' ability to produce embeddings.

2.3 Hashtag/Category Generation from Text

Hashtags and categories, as tagging mechanisms on social media, streamline the identification of topic-specific messages and aid users in finding themed content. Generating these from text can assist in creating identifiers for untagged notes or suggesting options to users based on their preferences. In this domain, there are three main methods: extractive, classification, and generative methods. Extractive methods identify key phrases in texts as hashtags or categories [61, 63], but cannot obtain those not present in the original text. Classification methods view this task as a text classification problem [14, 58, 60]. However, these may yield sub-optimal results due to the diverse, free-form nature of human-generated hashtags. Generative methods generate the hashtags/categories directly according to input texts [4, 44, 45]. Whereas, these methods are limited to solving the hashtag/category generation task. In NoteLLM, LLMs perform multi-task learning, simultaneously executing I2I recommendation and hashtag/category generation. Due to the similarity of these two tasks, learning to generate the hashtag/category can also enhance the I2I recommendation.

3 PROBLEM DEFINITION

In this section, we introduce the problem definition. We assume $\mathcal{N} = \{n_1, n_2, \dots, n_m\}$ as note pool, where m is the number of notes. Each note contains a title, hashtag, category, and content. We denote the i -th note as $n_i = (t_i, tp_i, c_i, ct_i)$, where t_i , tp_i , c_i , ct_i mean the title, the hashtag, the category and the content respectively. In the I2I note recommendation task, given a target note n_i , the LLM-based retriever aims to rank the top- k notes, which are similar to the given note, from the note pool $\mathcal{N} \setminus \{n_i\}$. In the hashtag/category generation task, the LLM is utilized to generate the hashtag tp_i according to t_i and ct_i . Besides, in the category generation task, the LLM is to generate the category c_i according to t_i , tp_i and ct_i .

4 METHODOLOGY

4.1 Framework of NoteLLM

In this subsection, we introduce the framework of NoteLLM, which comprises three key components: Note Compression Prompt Construction, GCL, and CSFT, as illustrated in Figure 2. We employ Note Compression Prompt to flexibly manage the I2I recommendation and hashtag/category generation tasks. These prompts are then tokenized and fed into LLMs. NoteLLM integrates both collaborative signals and semantic information into the hidden states. GCL uses the hidden states of the generated compressed word to conduct contrastive learning, thereby acquiring collaborative signals. Furthermore, CSFT leverages the semantic and collaborative information of the note to generate hashtags and categories.

4.2 Note Compression Prompt

We employ a unified Note Compression Prompt to facilitate both I2I recommendation and generation tasks. To leverage the generative capabilities of autoregressive LLMs for I2I recommendation

tasks [10], our aim is to condense the note content into a single special token. This condensed special token is then used to acquire collaborative knowledge through GCL. Subsequently, we generate hashtags/categories using this knowledge via CSFT.

Specifically, we propose the following prompt template for general note compression and hashtags/categories generation:

Prompt: [BOS]<Instruction> <Input Note> The compression word is:"[EMB]". <Output Guidance> <Output>[EOS]

In this template, [BOS], [EMB], and [EOS] are special tokens, while <Instruction>, <Input Note>, <Output Guidance>, and <Output> are placeholders replaced by specific content. The specific content for category generation is defined as follows:

Note Compression Prompt for Category Generation.
<Instruction>: Extract the note information in json format, compress it into one word for recommendation, and generate the category of the note.
<Input Note>: {'title': t_i , 'topic': tp_i , 'content': ct_i }.
<Output Guidance>: The category is:
<Output>: c_i

The template for hashtag generation is presented below:

Note Compression Prompt for Hashtag Generation.
<Instruction>: Extract the note information in json format, compress it into one word for recommendation, and generate <j> topics of the note.
<Input Note>: {'title': t_i , 'content': ct_i }.
<Output Guidance>: The <j> topics are:
<Output>: <j> topics from tp_i

Given the unpredictability of the number of hashtags generated by users, we randomly select a subset of original hashtags as the output target for hashtag generation to minimize potential misguidance to LLMs. The number of randomly selected hashtags, denoted as <j>, is incorporated into both the <Instruction> and <Output Guidance>.

Once the prompts are completed, they are tokenized and fed into the LLM. The LLM then distills the collaborative signals and key semantic information into the compressed word and generates hashtags/categories based on the central ideas of notes.

4.3 Generative-Contrastive Learning

Pre-trained LLMs usually learn new knowledge via instruction tuning [47, 62] or Reinforcement Learning from Human Feedback (RLHF) [32, 38]. These methods mainly focus on leveraging semantic information to enhance the effectiveness and safety of the LLMs. However, relying solely on semantic information in LLMs is insufficient for recommendation tasks [6, 20]. Collaborative signals, which are absent in LLMs, play a vital role in identifying the notes that are of specific interest to users [6, 20]. Therefore, we propose GCL to empower LLMs to capture stronger collaborative signals. In contrast to learning from specific answers or reward models, GCL adopts contrastive learning, which learns the relational proximity among notes from a holistic perspective.

In order to integrate collaborative signals into LLMs, we adopt the co-occurrence mechanism to construct the related note pairs based on user behaviours. This mechanism is based on the assumption that notes frequently read together are likely related. Therefore, we collect user behavior data within one week for the co-occurrence

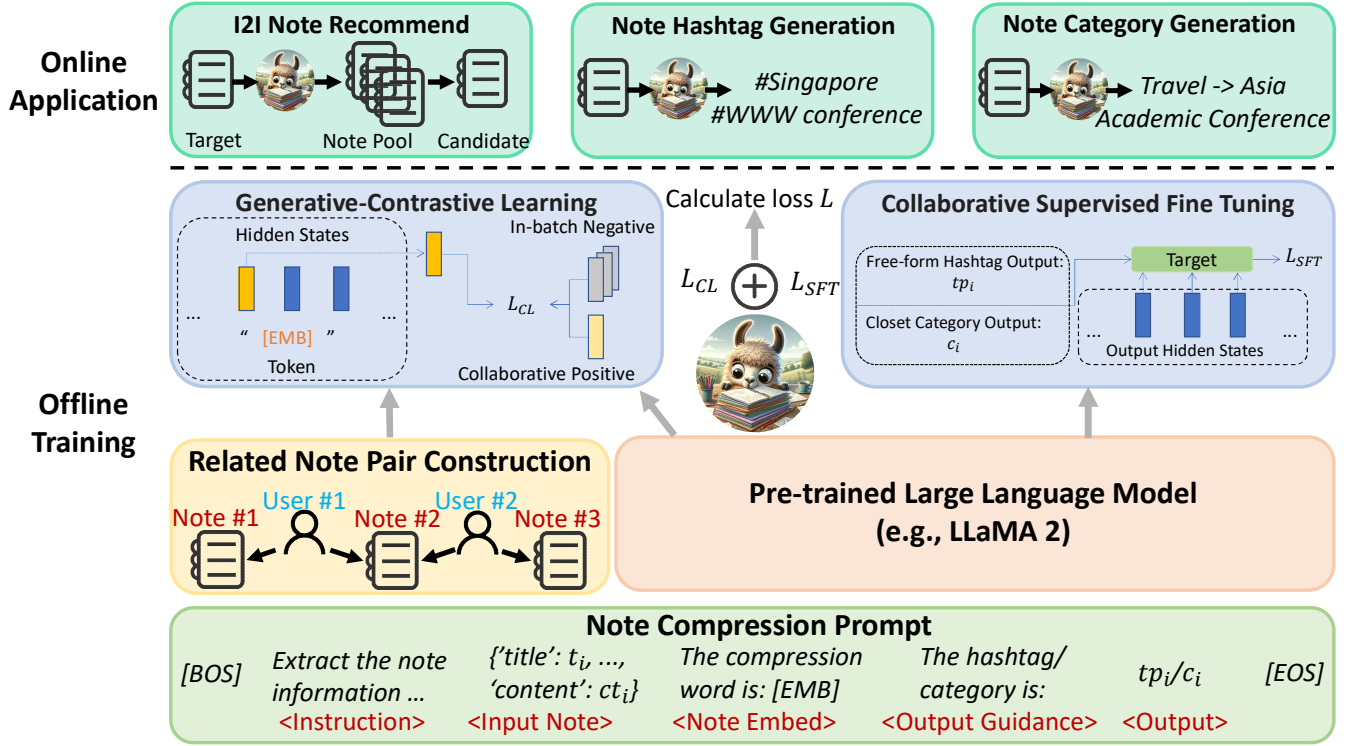


Figure 2: The NoteLLM framework uses a unified prompt for I2I note recommendations and hashtag/category generation. Notes are compressed via the Note Compression Prompt and processed by pre-trained LLMs. We utilize the co-occurrence mechanism to construct the related note pairs and train the I2I recommendation task using Generative-Contrasting Learning. NoteLLM also extracts note's key concepts for hashtag/category generation, enhancing the I2I recommendation task.

count. We count the occurrences in which users viewed note n_A and subsequently clicked on note n_B . Simultaneously, to differentiate the contribution of co-occurrence from different users, we assigned varying weights to distinct clicks. Specifically, we compute the co-occurrence score as following:

$$s_{n_A \rightarrow n_B} = \sum_{i=1}^U \frac{1}{N_i}, \quad (1)$$

where $s_{n_A \rightarrow n_B}$ represents the co-occurrence score from note n_A to note n_B , U is the number of users in this user behavior data, and N_i denotes the quantity of the note set clicked by the i -th user in the user behavior data. This operation aims to prevent the misdirection of active users, who might indiscriminately click on every note recommended to them. After calculating the co-occurrence score for all note pairs, we construct the set of co-occurrence scores S_{n_i} from note n_i to all other notes. Specifically, S_{n_i} is defined as $\{s_{n_i \rightarrow n_j} | 1 \leq j \leq m, i \neq j\}$. Next, we filter outlier notes whose co-occurrence scores are either above u or below the threshold l from S_{n_i} . Finally, we select the t notes with the highest co-occurrence scores from the filtered set as the related notes for note n_i .

After constructing the related notes pairs, we train NoteLLM to determine the relevance of notes based on textual semantics and collaborative signals. Different from simply taking a special pooling

word to represent the note [26], we utilize prompts to compress the note information to generate one virtual word. The last hidden state of the compressed virtual word contains the semantic information and collaborative signals of the given note, which can represent the note. Specifically, due to the autoregressive nature of LLMs, we take the last hidden state of the previous token of [EMB] and use a linear layer to transform it to note embedding space, whose dimension is d . We denote the embedding of i -th note n_i as $\mathbf{n}_i$. We assume each minibatch contains B related note pairs, resulting in a total of $2B$ notes per minibatch. We denote the related note of the note n_i as n_i^+ , and its embedding as $\mathbf{n}_i^+$. Following [30], the loss of GCL is computed as follows:

$$L_{cl} = -\frac{1}{2B} \sum_{i=1}^{2B} \log \frac{e^{\text{sim}(\mathbf{n}_i, \mathbf{n}_i^+) \cdot e^\tau}}{\sum_{j \in [2B] \setminus \{i\}} e^{\text{sim}(\mathbf{n}_i, \mathbf{n}_j) \cdot e^\tau}}, \quad (2)$$

where L_{cl} denotes the loss of GCL, τ means the learnable temperature and $\text{sim}(a, b) = a^\top b / (\|a\| \|b\|)$.

4.4 Collaborative Supervised Fine-Tuning

LLMs have gained prominence due to their robust capabilities in semantic understanding and generation. Several existing works attempt to apply the impressive abilities of LLMs to sentence embeddings [10, 26, 28, 30, 39]. However, these methods overlook the

generative capabilities of LLMs, reducing them to mere embedding generators and failing to fully exploit their potential. Besides, these methods underutilize hashtags/categories, which represent the key concepts of notes. In fact, generating hashtags/categories is similar to producing note embeddings. Both tasks aim to summarize note content. The task of generating hashtags/categories extracts key note information from a text generation perspective, while the task of producing note embeddings compresses notes into a virtual word from a collaborative viewpoint for I2I recommendation. To this end, our NoteLLM jointly models the GCL and CSFT tasks to potentially enhance the quality of embeddings. We integrate these two tasks into a single prompt, providing additional information for both tasks and streamlining the training process.

Specifically, we adopt CSFT, which leverages the semantic content of the notes and the collaborative signals in the compressed token to generate hashtags/categories. To enhance training efficiency and prevent the forgetting problem [40], we select r notes from each batch for the hashtag generation task, while the remaining notes are allocated for the category generation task. Specifically, we compute the CSFT loss as follows:

$$L_{gen} = -\frac{1}{T} \sum_{i=1}^T \log(p(o_i | o_{<i}, i)), \quad (3)$$

where L_{gen} is the CSFT loss, T is the length of the output, o_i means the i -th token in output sequence o and i is the input sequence.

Finally, we define the loss function of NoteLLM to incorporate both GCL and CSFT, as follows:

$$L = \frac{L_{cl} + \alpha L_{gen}}{1 + \alpha}, \quad (4)$$

where L is the total loss of NoteLLM and α is the hyperparameter. Through model updates, NoteLLM is capable of concurrently executing I2I recommendation tasks and hashtag/category generation tasks for note recommendation scenarios.

5 EXPERIMENTS

5.1 Dataset and Experiment Setting

Table 1: Detailed statistics of training and testing dataset.

training dataset			
#notes	458,221	#note pairs	312,564
avg. #words per title	11.54	avg. #hashtag per note	3.02
avg. #words per hashtag	4.19	avg. #words per content	47.67
testing dataset			
#notes	257,937	#note pairs	27,999
avg. #words per title	13.70	avg. #hashtag per note	5.49
avg. #words per hashtag	4.53	avg. #words per content	182.45

We conduct offline experiments on Xiaohongshu product datasets. To balance the model’s training, we generate the training set by extracting a fixed number of note pairs from each category combination based on a week’s product data. Then, we randomly select notes from the upcoming month to form the note pool of testing set, excluding any notes that are already in the training dataset. The detailed statistics of the training and testing dataset are shown in Table 1. Besides, there are more than 500 categories in our dataset.

In our experiments, we leverage Meta LLaMA 2 [42] as the base LLMs. In related note pair construction, we set the upper bound of the co-occurrence score u as 30 and the lower bound l as 0.01. And we set t as 10. Besides, the dimension d of note embedding is set to 128. The batch size B is set to 64^1 . Each batch contains 128 notes. Due to context length restriction, we truncate the titles exceeding 20 tokens, and truncate the contents exceeding 80 tokens. The temperature τ is initialized as 3. We set α in Equation 4 to 0.01. The ratio r for the hashtag generation task is set at 40%.

To assess the offline performance of the I2I recommendation model, we choose the prompt for category generation, which contains all input note information. We select the first note from each note pair as the target note, and the other as the ground truth. Subsequently, we rank all the notes in the test pool, excluding the target note, according to the target note. We then use Recall@100, Recall@1k, Recall@10k and Recall@100k to validate the model effectiveness for I2I note recommendation. For closet category generation tasks, we use accuracy (Acc.) and illusory proportions (Ill.) as the evaluation metrics. Ill. represent the proportion of categories generated by the model that are not in the closet. For free-form hashtag generation tasks, we use BLEU4, ROUGE1, ROUGE2 and ROUGEL to evaluate models.

5.2 Offline Performance Evaluation

In this subsection, we demonstrate the effectiveness of NoteLLM for I2I note recommendation. We compare our NoteLLM with the following text-based I2I recommendation methods:

- **zero-shot** utilizes LLMs to generate the embeddings without any prompts and then conducts zero-shot retrieval.
- **PromptEOL zero-shot** [10] is a zero-shot LLMs sentence embedding method that uses the explicit one-word limitation prompt.
- **SentenceBERT** [36] adopts BERT to learn the note similarity based on contrastive learning, serving as the online baseline.
- **PromptEOL+CSE** [10] uses the explicit one-word limitation prompt and leverages contrastive learning to update LLMs.
- **RepLLaMA** [26], a bi-encoder dense retriever based on LLMs without any prompts.

The results, as presented in Table 2, offer several insightful observations. Despite their potential, zero-shot methods are still unable to surpass the performance of fine-tuned methods, suggesting that the latter’s specific knowledge in the note recommendation domain gives them an edge. Further, we find that the comparison between methods based on LLaMA 2 and SentenceBERT reveals a significant advantage for the former, indicating a superior ability of LLMs to understand notes. The performance of PromptEOL+CSE with specific prompts matches that of RepLLaMA without prompts, indicating that prompts boost zero-shot retrieval but their effect lessens after fine-tuning. Lastly, our NoteLLM outperforms other LLM-based methods, primarily due to CSFT’s effective transfer of summary ability into note embedding compression, which efficiently distills key points for improved note embeddings.

¹We utilize Distributed Data Parallel training on $8 \times 80\text{GB}$ Nvidia A100 GPUs and the batch size per each GPU is 8.

Table 2: Performance of different methods in I2I recommendation tasks (%).

	Model Size	Recall@100	Recall@1k	Recall@10k	Recall@100k	Avg.
LLaMA 2 zero-shot	7B	11.94	19.44	32.53	68.81	33.18
PromptEOL zero-shot [10]	7B	55.27	74.47	88.71	98.04	79.12
SentenceBERT (Online) [36]	110M	70.72	87.88	96.29	99.62	88.63
PromptEOL+CSE [10]	7B	83.28	95.26	99.20	99.96	94.43
RepLLaMA [26]	7B	83.63	95.10	99.27	99.94	94.49
NoteLLM	7B	84.02	95.23	99.23	99.96	94.66

Table 3: Performance of different methods for low exposure notes and high exposure notes in I2I recommendation tasks (%).

	Low Exposure		High Exposure		Overall	
	Recall@100	Recall@1k	Recall@100	Recall@1k	Recall@100	Recall@1k
SentenceBERT (Online) [36]	75.00	90.54	59.03	81.91	70.72	87.88
PromptEOL+CSE [10]	86.28	96.63	72.46	91.40	83.28	95.26
RepLLaMA [26]	86.54	96.18	72.64	91.37	83.63	95.10
NoteLLM	87.85	96.63	73.46	91.26	84.02	95.23

5.3 Effect on Different Exposure Notes

In this subsection, we demonstrate the efficacy of our NoteLLM in handling notes with varying levels of exposure. For a more comprehensive analysis, we have divided the ground truth notes into two distinct categories based on their exposure levels. The first category encompasses notes with low-exposure, specifically those with an exposure of less than 1, 500. Despite constituting 30% of all test notes, their cumulative exposure only amounts to 0.5%. On the other hand, the second category includes notes with high-exposure, characterized by an exposure exceeding 75, 000. Even though they represent only 10% of all test notes, their collective exposure is substantial, accounting for 75% of the total. We then separately calculate the recall for these two groups to further understand the performance of our NoteLLM across different exposure levels.

The results are presented in Table 3. NoteLLM consistently outperforms other methods for both low and high exposure notes in most cases, which indicates that the incorporation of CSFT module provides consistent benefits across all notes, irrespective of their exposure levels. It’s worth noting that while these methods exhibit commendable performance with low-exposure notes, they falter when dealing with high-exposure notes. The decline in performance can be attributed to neglecting the popularity bias [17]. Such properties enhance the model’s ability to recall based on the content of the notes, making it particularly suitable for retrieving cold-start notes. This can motivate users to post more new notes, creating richer content for the entire community.

5.4 Ablation Study

In this subsection, we conduct an ablation study to underscore the effectiveness of the key innovations in our work. To enhance our analysis, we also show performance on the category and hashtag generation tasks in the following experiments.

We compare our NoteLLM with following variants:

- **w/o CSFT**, a method solely employs the GCL module.
- **w/o GCL** ($r = 40\%$) only adopts CSFT module to guide LLMs in summarizing the hashtags and categories.
- **w/o GCL** ($r = 0\%$) only has category summary task.
- **w/o GCL** ($r = 100\%$) only instructs LLMs to summarize hashtags.

The results are presented in Table 4, from which we can draw several conclusions. Firstly, we observe that the ablation without CSFT performs worse than NoteLLM in I2I recommendation task, and completely loses the ability to generate hashtags and categories. This highlights the crucial role of the CSFT module in enhancing note embeddings and suggests that a single model can handle both recommendation and generation tasks. Secondly, we find that the ablation without GCL module outperforms PromptEOL zero-shot in I2I recommendation tasks. This indicates that the hashtag/category generation task can enhance I2I recommendation tasks. Thirdly, the ablation without GCL ($r = 40\%$) performs better in I2I recommendation tasks than the version without GCL ($r = 0\%$) and without GCL ($r = 100\%$). This suggests that task diversity is important for CSFT [22]. Finally, we observe a clear seesaw phenomenon [66] for hashtag and category generation tasks. The ablation without GCL ($r = 0\%$) can generate correct categories for 80.64% of notes, but struggles to summarize useful hashtags. Conversely, the version without GCL ($r = 100\%$) generates high-quality hashtags, but the generated categories are mostly incorrect.

5.5 Impact of Data Diversity in CSFT Module

In this subsection, we investigate the impact of data diversity in CSFT module. The performance of the model for different tasks under varying data type proportions is presented in Table 5.

For I2I recommendation tasks, performance improves as r increases. This is attributed to the enhanced data diversity of instruction tuning with a higher r , which more effectively instructs LLMs to summarize and compress various types of information.

Table 4: Ablation study for NoteLLM in I2I recommendation, category generation and hashtag generation tasks (%).

Model	R@100	R@1k	R@10k	R@100k	Avg.	Acc.	Ill.	BLEU4	ROUGE1	ROUGE2	ROUGEL
NoteLLM	84.02	95.23	99.23	99.96	94.66	66.17	0.50	1.38	22.31	8.02	21.03
w/o CSFT	83.28	95.26	99.20	99.96	94.43	0.00	100.00	0.01	0.00	0.00	0.00
w/o GCL ($r = 40\%$)	75.38	90.33	97.09	98.93	90.43	75.12	2.27	1.28	26.50	13.02	23.27
w/o GCL ($r = 0\%$)	60.38	83.22	96.13	98.84	84.64	80.64	0.07	0.00	1.54	0.00	1.54
w/o GCL ($r = 100\%$)	71.98	87.86	95.59	98.51	88.49	0.18	99.70	1.30	27.66	14.19	24.11

Table 5: Performance of NoteLLM under different data diversity in CSFT module for I2I recommendation, category generation and hashtag generation tasks (%).

r	R@100	R@1k	R@10k	R@100k	Avg.	Acc.	Ill.	BLEU4	ROUGE1	ROUGE2	ROUGEL
0%	83.29	95.07	99.14	99.96	94.37	71.00	0.12	0.00	0.00	0.00	0.00
20%	83.81	95.28	99.26	99.96	94.58	69.70	0.09	1.59	22.57	7.73	21.64
40%	84.02	95.23	99.23	99.96	94.66	66.17	0.50	1.38	22.31	8.02	21.03
60%	83.37	95.03	99.25	99.96	94.40	63.37	0.90	1.33	21.92	7.88	20.62
80%	83.15	95.06	99.27	99.97	94.36	53.60	2.67	1.34	22.48	8.31	21.10
100%	82.49	94.49	99.11	99.96	94.01	0.00	100.00	1.33	21.88	7.99	20.47

Table 6: Performance of NoteLLM under varying CSFT module magnitudes for I2I recommendation, category generation and hashtag generation tasks (%).

Model	R@100	R@1k	R@10k	R@100k	Avg.	Acc.	Ill.	BLEU4	ROUGE1	ROUGE2	ROUGEL
$\alpha = 0$	83.42	95.13	99.31	99.96	94.46	0.00	100.00	0.05	0.91	0.01	0.87
$\alpha = 0.001$	83.73	95.09	99.26	99.97	94.51	49.04	6.84	1.82	15.13	3.23	14.80
$\alpha = 0.01$	84.02	95.23	99.23	99.96	94.66	66.17	0.50	1.38	22.31	8.02	21.03
$\alpha = 0.1$	83.51	95.18	99.31	99.96	94.49	73.33	0.04	1.33	24.44	10.42	22.28
$\alpha = 1$	83.79	94.79	99.26	99.97	94.45	74.69	0.82	1.33	26.94	13.21	23.90
$\alpha = 10$	82.92	94.28	98.94	99.96	94.03	75.15	2.08	1.30	27.63	13.95	24.21

However, as r continues to increase and the data for instruction tuning becomes more skewed towards the hashtag generation task, performance begins to decline. For category generation tasks, as the data becomes more biased towards the hashtag generation task, the performance on category generation deteriorates. However, as r continues to increase from 20%, there is not much significant change in the hashtag generation task. This may be because the category generation task is a closet task, which requires a stringent match. In contrast, the hashtag generation task is a free-form generation task, which allows for greater flexibility.

5.6 Impact of the Magnitude of CSFT Module

In this subsection, we explore the impact of the magnitude of CSFT module on task performance. We present the results of our experiments in Table 6. Our findings suggest that a slight increase in α enhances the performance of both recommendation and generation tasks. However, as α continues to increase, the performance of the recommendation task begins to decline, while the performance of the generation task continues to improve. This reveals a trade-off

between generation and I2I recommendation tasks, highlighting the need for a balanced approach.

5.7 Case Study

Finally, we show some cases for note recommendation and generation tasks as shown in Figure 3. In Figure 3(a), the query note suggests which summer clothes to avoid buying, while all baselines recommend summer outfits. NoteLLM can accurately recommend notes that are related to simple living. In Figure 3(b), baselines misinterpret 'rabbit' in the note as a live rabbit, instead of the toy rabbit from *Keep strangers away*. Figure 3(c) and Figure 3(d) show the cases for hashtag generation tasks, which shows the benefits of NoteLLM. RedHashtag is an online hashtag generation method for Xiaohongshu, which is based on classification from the fixed hashtag set. In Figure 3(c), NoteLLM is not deceived by the semantic information 'factories'. Instead, NoteLLM correctly identifies that the note's content primarily focuses on taking photos. In Figure 3(d), NoteLLM is capable of generating more specific and long-tail hashtags, as opposed to the more generic ones. However, our method still suffers from the hallucination problem [57].

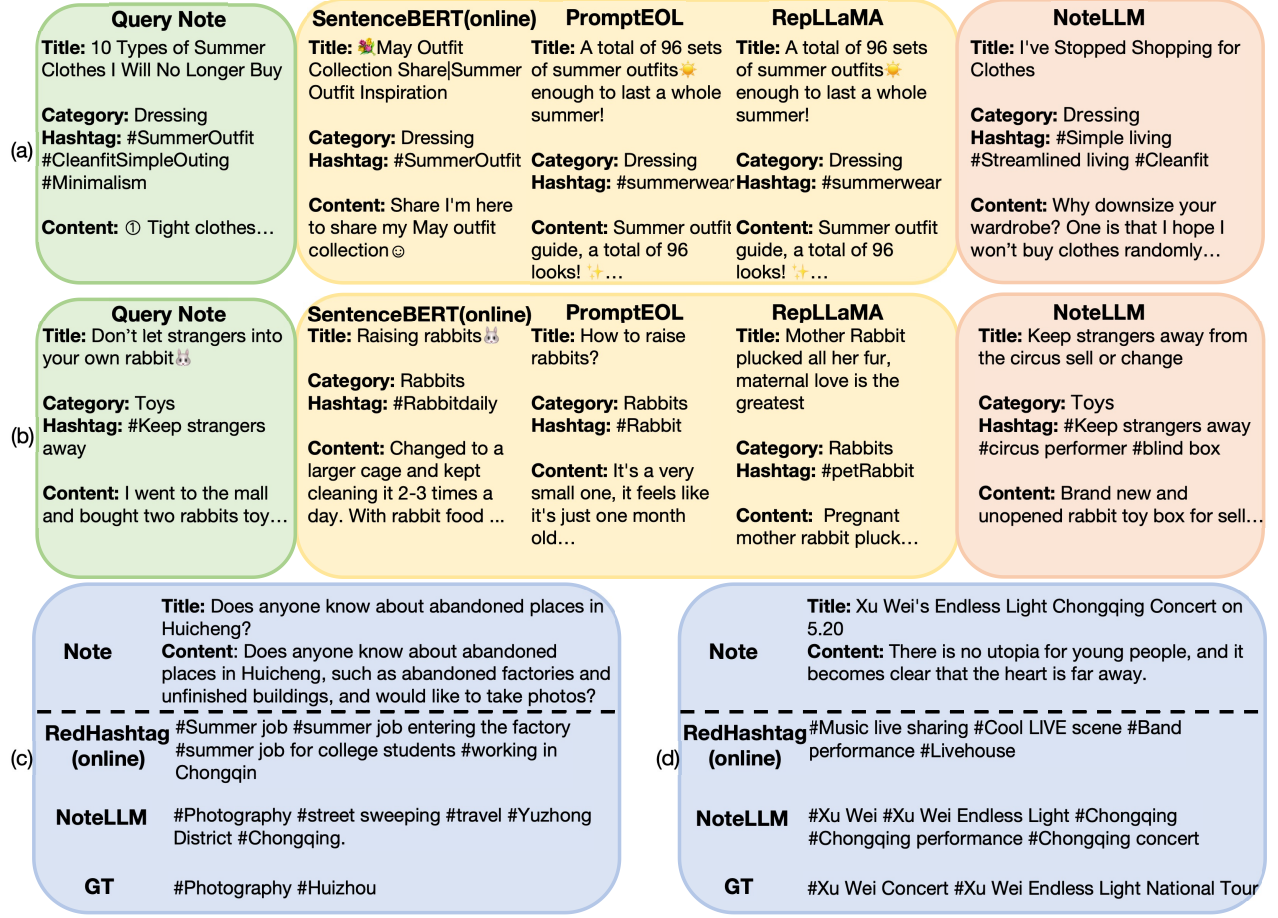


Figure 3: The visualization cases of NoteLLM and other baselines. Figure 3(a) and 3(b) show the cases in I2I recommendation tasks, where the left query note is the user's clicked note, and the remaining notes are the top-1 ranked results retrieved by different methods. Figure 3(c) and 3(d) show the cases in hashtag generation tasks. RedHashtag is the online hashtag generation method. GT means the ground truth hashtags.

5.8 Online Experiments

We conduct week-long online I2I recommendation experiments on Xiaohongshu. Compared to the previous online method that adopts SentenceBERT, our NoteLLM improves the click-through rate by 16.20%. Furthermore, the enhanced recall performance increases the number of comments by 1.10% and the average weekly number of publishers (WAP) by 0.41%. These results indicate the introduction of LLMs into I2I note recommendation tasks can improve recommendation performance and user experience. Besides, we observe a noteworthy increase of 3.58% in the number of comments on new notes within a single day. This denotes the generalization of LLMs is beneficial to cold start notes. Now, we have deployed our NoteLLM into the I2I note recommendation task on Xiaohongshu.

6 CONCLUSION

In this work, we propose retrievable LLMs, called NoteLLM, for note recommendation with three key components: Note Compression Prompt, GCL, and CSFT. To manage both I2I recommendation and

hashtag/category generation tasks, we utilize Note Compression Prompt to form the compressed word embeddings and generate the hashtag/category simultaneously. Then, we use GCL to conduct contrastive learning based on the hidden states of the compressed word, which acquires collaborative signals. Additionally, we employ CSFT to preserve the generation capability of NoteLLM while leveraging the semantic and collaborative information of the note to generate hashtags and categories, which can enhance the embeddings for recommendation. Comprehensive experiments are conducted, which validate the effectiveness of NoteLLM.

ACKNOWLEDGEMENTS

This work was supported in part by the grants from National Natural Science Foundation of China (No.62222213, U22B2059, 62072423), and the USTC Research Funds of the Double First-Class Initiative (No.YD2150002009).

REFERENCES

- [1] Keqin Bao, Jizhi Zhang, Wenjie Wang, Yang Zhang, Zhengyi Yang, Yancheng Luo, Fuli Feng, Xiangnan He, and Qi Tian. 2023. A bi-step grounding paradigm for large language models in recommendation systems. *arXiv preprint arXiv:2308.08434* (2023).
- [2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. In *RecSys*.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [4] Shizhe Diao, Sedrick Scott Keh, Liangming Pan, Zhiliang Tian, Yan Song, and Tong Zhang. 2023. Hashtag-Guided Low-Resource Tweet Classification. In *WWW*. 1415–1426.
- [5] Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, and Qing Li. 2023. Recommender systems in the era of large language models (llms). *arXiv preprint arXiv:2307.02046* (2023).
- [6] Zhankui He, Zhouhang Xie, Rahul Jha, Harald Steck, Dawen Liang, Yesu Feng, Bodhisattwa Prasad Majumder, Nathan Kallus, and Julian McAuley. 2023. Large language models as zero-shot conversational recommenders. In *CIKM*. 720–730.
- [7] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2023. Large language models are zero-shot rankers for recommender systems. *arXiv preprint arXiv:2305.08845* (2023).
- [8] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *CIKM*. 2333–2338.
- [9] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2023. Recommender AI Agent: Integrating Large Language Models for Interactive Recommendations. *arXiv preprint arXiv:2308.16505* (2023).
- [10] Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. 2023. Scaling Sentence Embeddings with Large Language Models. *arXiv preprint arXiv:2307.16645* (2023).
- [11] Ting Jiang, Jian Jiao, Shaohan Huang, Zihan Zhang, Deqing Wang, Fuzhen Zhuang, Furu Wei, Haizhen Huang, Deny Deng, and Qi Zhang. 2022. Prompt-BERT: Improving BERT Sentence Embeddings with Prompts. In *EMNLP*. 8826–8837.
- [12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [13] Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906* (2020).
- [14] Fei-Fei Kou, Jun-Ping Du, Cong-Xian Yang, Yan-Song Shi, Wan-Qiu Cui, Mei-Yu Liang, and Yue Geng. 2018. Hashtag recommendation based on multi-features of microblogs. *JCSST* 33 (2018), 711–726.
- [15] Ruyu Li, Wenhao Deng, Yu Cheng, Zheng Yuan, Jiaqi Zhang, and Fajie Yuan. 2023. Exploring the Upper Limits of Text-Based Collaborative Filtering Using Large Language Models: Discoveries and Insights. *arXiv preprint arXiv:2305.11700* (2023).
- [16] Mengqi Liao, S Shyam Sundar, and Joseph B. Walther. 2022. User trust in recommendation systems: A comparison of content-based, collaborative and demographic filtering. In *CHI*. 1–14.
- [17] Allen Lin, Jianling Wang, Ziwei Zhu, and James Caverlee. 2022. Quantifying and mitigating popularity bias in conversational recommender systems. In *CIKM*. 1238–1247.
- [18] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Hui Feng Guo, Yong Yu, Ruiming Tang, et al. 2023. How Can Recommender Systems Benefit from Large Language Models: A Survey. *arXiv preprint arXiv:2306.05817* (2023).
- [19] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet computing* 7, 1 (2003), 76–80.
- [20] Junling Liu, Chao Liu, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is chatgpt a good recommender? a preliminary study. *arXiv preprint arXiv:2304.10149* (2023).
- [21] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2023. A First Look at LLM-Powered Generative News Recommendation. *arXiv preprint arXiv:2305.06566* (2023).
- [22] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V Le, Barret Zoph, Jason Wei, et al. 2023. The flan collection: Designing data and methods for effective instruction tuning. *arXiv preprint arXiv:2301.13688* (2023).
- [23] Hanjia Lyu, Song Jiang, Hanqing Zeng, Yinglong Xia, and Jiebo Luo. 2023. Llm-rec: Personalized recommendation via prompting large language models. *arXiv preprint arXiv:2307.15780* (2023).
- [24] Yuanjie Lyu, Zhiyu Li, Simin Niu, Feiyu Xiong, Bo Tang, Wenjin Wang, Hao Wu, Huanyong Liu, Tong Xu, and Enhong Chen. 2024. CRUD-RAG: A Comprehensive Chinese Benchmark for Retrieval-Augmented Generation of Large Language Models. *arXiv preprint arXiv:2401.17043* (2024).
- [25] Xinyu Ma, Jiafeng Guo, Ruqing Zhang, Yixing Fan, and Xueqi Cheng. 2022. Pre-train a discriminative text encoder for dense retrieval via contrastive span prediction. In *SIGIR*. 848–858.
- [26] Xueguang Ma, Liang Wang, Nan Yang, Furu Wei, and Jimmy Lin. 2023. Fine-Tuning LLaMA for Multi-Stage Text Retrieval. *arXiv preprint arXiv:2310.08319* (2023).
- [27] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781* (2013).
- [28] Niklas Muennighoff. 2022. Sgpt: Gpt sentence embeddings for semantic search. *arXiv preprint arXiv:2202.08904* (2022).
- [29] Sheshera Mysore, Andrew McCallum, and Hamed Zamani. 2023. Large Language Model Augmented Narrative Driven Recommendations. *arXiv preprint arXiv:2306.02250* (2023).
- [30] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. 2022. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005* (2022).
- [31] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [32] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *NeurIPS* 35 (2022), 27730–27744.
- [33] Wenjun Peng, Guiyang Li, Yue Jiang, Zilong Wang, Dan Ou, Xiaoyi Zeng, Enhong Chen, et al. 2023. Large Language Model based Long-tail Query Rewriting in Taobao Search. *arXiv preprint arXiv:2311.03758* (2023).
- [34] Wenjun Peng, Derong Xu, Tong Xu, Jianjin Zhang, and Enhong Chen. 2023. Are gpt embeddings useful for ads and recommendation?. In *KSEM*. Springer, 151–162.
- [35] Juan Ramos et al. 2003. Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning*, Vol. 242. Citeseer, 29–48.
- [36] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP-IJCNLP*.
- [37] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [38] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. 2020. Learning to summarize with human feedback. *NeurIPS* 33 (2020), 3008–3021.
- [39] Hongjin Su, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A Smith, Luke Zettlemoyer, Tao Yu, et al. 2023. One embedder, any task: Instruction-finetuned text embeddings. *ACL Findings* (2023).
- [40] Mariya Toneva, Alessandro Sordani, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J Gordon. 2018. An empirical study of example forgetting during deep neural network learning. *arXiv preprint arXiv:1812.05159* (2018).
- [41] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [42] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrutli Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [43] Yancheng Wang, Ziyang Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojian Huang, Yanbin Lu, and Yingzhen Yang. 2023. Recmind: Large language model powered agent for recommendation. *arXiv preprint arXiv:2308.14296* (2023).
- [44] Yue Wang, Jing Li, Hou Pong Chan, Irwin King, Michael R Lyu, and Shuming Shi. 2019. Topic-Aware Neural Keyphrase Generation for Social Media Language. In *ACL*. 2516–2526.
- [45] Yue Wang, Jing Li, Irwin King, Michael R Lyu, and Shuming Shi. 2019. Microblog hashtag generation via encoding conversation contexts. *arXiv preprint arXiv:1905.07584* (2019).
- [46] Zifeng Wang, Chufan Gao, Cao Xiao, and Jimeng Sun. 2023. AnyPredict: Foundation Model for Tabular Prediction. *arXiv preprint arXiv:2305.12081* (2023).
- [47] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652* (2021).
- [48] Chuhan Wu, Fangzhao Wu, Yongfeng Huang, and Xing Xie. 2023. Personalized news recommendation: Methods and challenges. *ACM Transactions on Information Systems* 41, 1 (2023), 1–50.
- [49] Chuhan Wu, Fangzhao Wu, Tao Qi, and Yongfeng Huang. 2021. Empowering news recommendation with pre-trained language models. In *SIGIR*. 1652–1656.
- [50] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2023. A Survey on Large Language Models for Recommendation. *arXiv preprint arXiv:2305.19860* (2023).

- [51] Yunjia Xi, Weiwen Liu, Jianghao Lin, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, Rui Zhang, and Yong Yu. 2023. Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models. *arXiv preprint arXiv:2306.10933* (2023).
- [52] Shitao Xiao, Zheng Liu, Weihao Han, Jianjin Zhang, Yingxia Shao, Defu Lian, Chaozhuo Li, Hao Sun, Denvy Deng, Liangjie Zhang, et al. 2022. Progressively optimized bi-granular document representation for scalable embedding based retrieval. In *WWW*. 286–296.
- [53] Shitao Xiao, Zheng Liu, Yingxia Shao, Tao Di, Bhuvan Middha, Fangzhao Wu, and Xing Xie. 2022. Training large-scale news recommenders with pretrained language models in the loop. In *KDD*. 4215–4225.
- [54] Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, and Enhong Chen. 2023. Large Language Models for Generative Information Extraction: A Survey. *arXiv preprint arXiv:2312.17617* (2023).
- [55] Xiaoyong Yang, Yadong Zhu, Yi Zhang, Xiaobo Wang, and Quan Yuan. 2020. Large scale product graph construction for recommendation in e-commerce. *arXiv preprint arXiv:2010.05525* (2020).
- [56] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2023. A Survey on Multimodal Large Language Models. *arXiv preprint arXiv:2306.13549* (2023).
- [57] Shukang Yin, Chaoyou Fu, Sirui Zhao, Tong Xu, Hao Wang, Dianbo Sui, Yunhang Shen, Ke Li, Xing Sun, and Enhong Chen. 2023. Woodpecker: Hallucination correction for multimodal large language models. *arXiv preprint arXiv:2310.16045* (2023).
- [58] Jichuan Zeng, Jing Li, Yan Song, Cuiyun Gao, Michael R Lyu, and Irwin King. 2018. Topic Memory Networks for Short Text Classification. In *EMNLP*. 3120–3131.
- [59] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001* (2023).
- [60] Qi Zhang, Jiawen Wang, Haoran Huang, Xuanjing Huang, and Yeyun Gong. 2017. Hashtag Recommendation for Multimodal Microblog Using Co-Attention Network. In *IJCAI*. 3420–3426.
- [61] Qi Zhang, Yang Wang, Yeyun Gong, and Xuan-Jing Huang. 2016. Keyphrase extraction using deep recurrent neural networks on twitter. In *EMNLP*. 836–845.
- [62] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. 2023. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792* (2023).
- [63] Yingyi Zhang, Jing Li, Yan Song, and Chengzhi Zhang. 2018. Encoding conversation context for neural keyphrase extraction from microblog posts. In *NAACL*. 1676–1686.
- [64] Weihao Zhao, Han Wu, Weidong He, Haoyang Bi, Hao Wang, Chen Zhu, Tong Xu, and Enhong Chen. 2023. Hierarchical Multi-modal Attention Network for Time-sync Comment Video Recommendation. *IEEE Transactions on Circuits and Systems for Video Technology* (2023).
- [65] Xinpeng Zhao, Ying Zhang, Qiang Xiao, Yuming Ren, and Yingchun Yang. 2023. Bootstrapping Contrastive Learning Enhanced Music Cold-Start Matching. In *WWW*. 351–355.
- [66] Shen Zheng, Yuyu Zhang, Yijie Zhu, Chengguang Xi, Pengyang Gao, Xun Zhou, and Kevin Chen-Chuan Chang. 2023. GPT-Fathom: Benchmarking Large Language Models to Decipher the Evolutionary Path towards GPT-4 and Beyond. *arXiv preprint arXiv:2309.16583* (2023).
- [67] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *KDD*. 1079–1088.